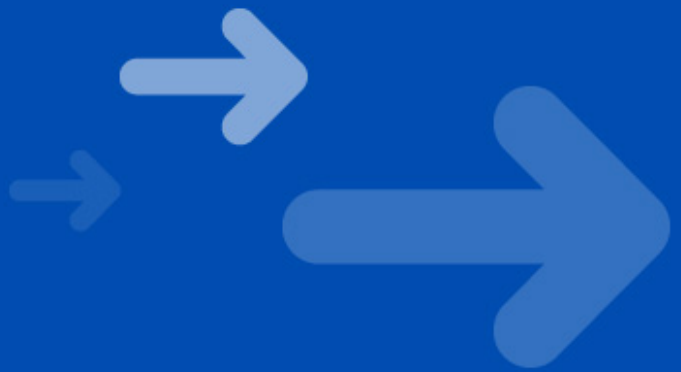
Symbian OS Communications

Phong Vu

November 23, 2004



Get Connected

Content

Symbian OS – Communications overview

Serial communications

Socket based communication

Communications Database

Bluetooth

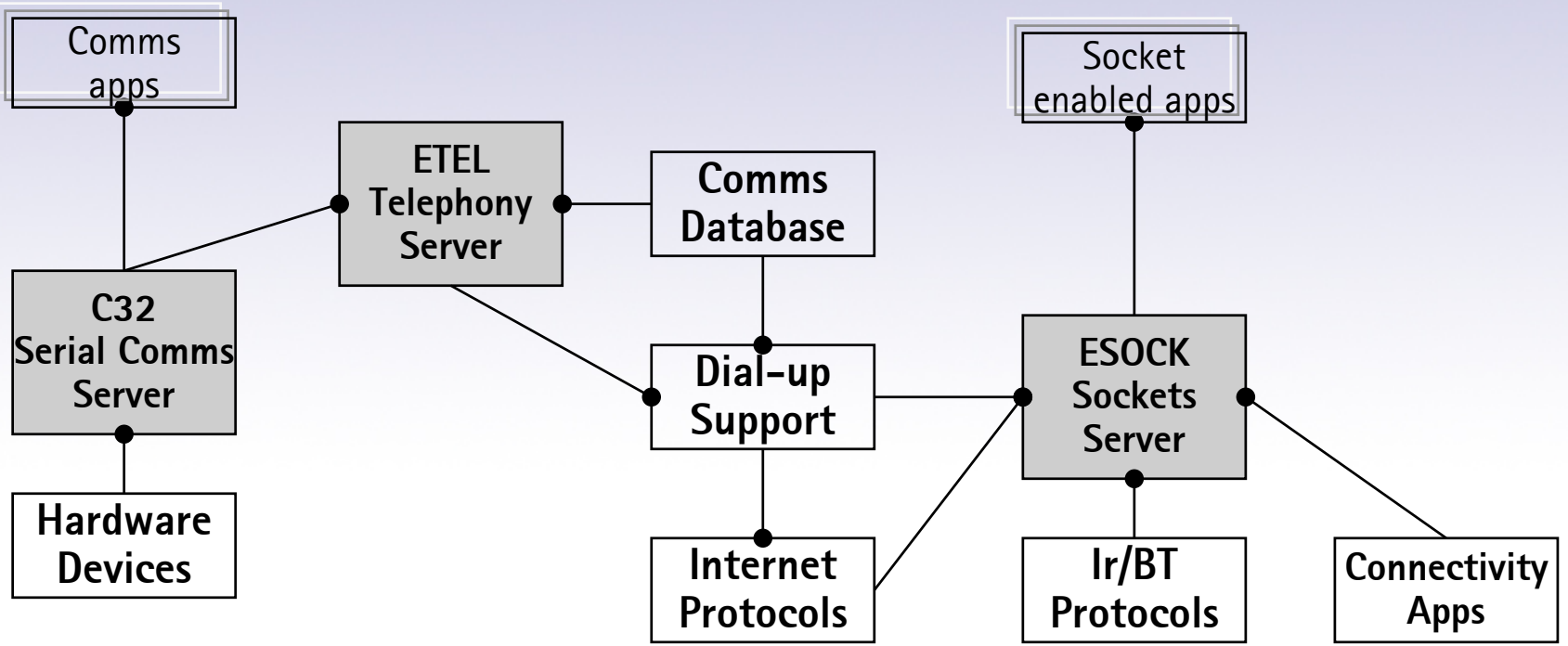
More information

Get Connected

Communication Overview

- Client/server architecture
- Serial communications
- Socket based communication
- Telephony communication

Communication Overview



- C32 – the serial communications server, which drives the communications ports and also runs serial-line protocols that use other communications services
- ESOCK – the sockets server, which provides a socket-based API that is used for dial-up TCP/IP, IrDA and Bluetooth protocol
- ETEL – the telephony server, which provides services to implement data and voice calls.

Content

Symbian OS – Communications overview

Serial communications

Socket based communication

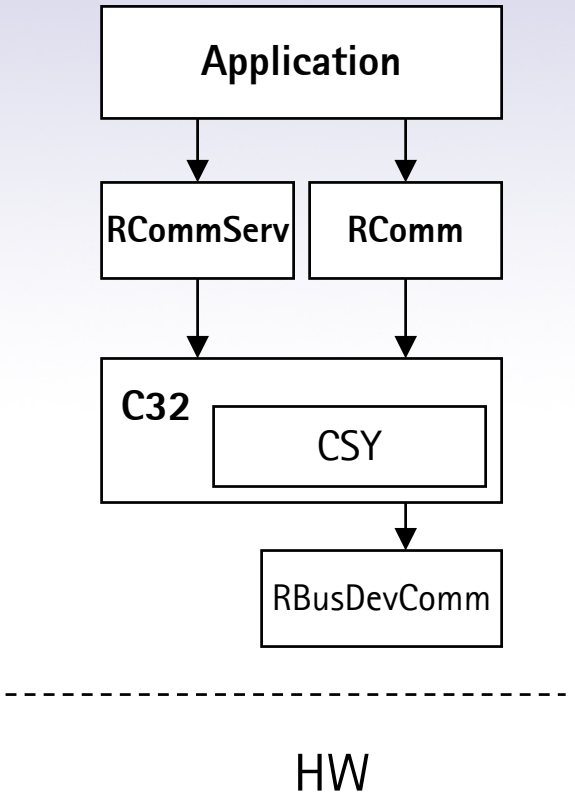
Communications Database

Bluetooth

More information

Serial Communications Overview

- Point-to-point communication
- Short range communication over infrared or Bluetooth
- C32 provides RCommServ and RComm API
  - RCommServ sets up the port (initiates)
  - RComm provides the actual port interface



Serial Communications

- Load Physical and Logical Device Driver
  - For WINS emulator

```
User::LoadPhysicalDevice(_L("ECDRV"));
```

  - For target build

```
User::LoadPhysicalDevice(_L("EUART1"));
```

```
User::LoadLogicalDevice(_L("ECOMM"));
```

- Start the Comms server C32

```
StartC32();
```

- Create a Comms server handle and connect to the Comms server

```
RCommServ server;
TInt ret = commServer.Connect();
```

- Load a comms module
  - RS-232 module: `ECUART`; Infrared module: `IRCOMM`; Bluetooth module: `BTCOMM`

```
TInt ret= commServer.LoadCommModule(_L("IRCOMM"));
```

Serial Communications

- Create and open a serial port
  - Port name: RS-232 -> COMM::x; Infrared -> IRCOMM::; Bluetooth -> BTCOMM::

```
RComm comPort;
TInt ret= commPort.Open(commServer, _L("IRCOMM::0"), ECommExclusive);
```

- Configure the serial port
  - Read port's capabilities

```
TCommCaps2 portCaps;
commPort.Caps(portCaps);
```

  - Change port's settings

```
TCommConfig portSettings;
commPort.Config(portSettings);                  // read current port settings
portSettings().iRate = EBps19200;               // set data rate in bit per second
portSettings().iHandshake = KConfigFailDSR; // set handshaking control - no handshaking
commPort.SetConfig(portSettings);
```

  - Set other settings

```
commPort.SetSignal(KSignalDTR, 0);
commPort.SetSignal(KSignalRTS, 0);
```

Serial Communications

- Transferring data over the port
  - Reading

```
commPort.Read(TRequestStatus &aStatus, TDes8 &aDes);
commPort.Read(TRequestStatus &aStatus, TDes8 &aDes, TInt aLength);
commPort.Read(TRequestStatus &aStatus, TTimeIntervalMricroSeconds32 aTimeOut,
  TDes8 &aDes);
commPort.Read(TRequestStatus &aStatus, TTimeIntervalMricroSeconds32 aTimeOut,
  TDes8 &aDes, TInt aLength);
commPort.ReadOneOrMore(TRequestStatus &aStatus, TDes8 &aDes);
```

  - Writing

```
commPort.Write(TRequestStatus &aStatus, const TDesC8 &aDes);
commPort.Write(TRequestStatus &aStatus, const TDesC8 &aDes, TInt aLength);
commPort.Write(TRequestStatus &aStatus, TTimeIntervalMricroSeconds32 aTimeOut, const
  TDesC8 &aDes);
commPort.Write(TRequestStatus &aStatus, TTimeIntervalMricroSeconds32 aTimeOut, const
  TDesC8 &aDes, TInt aLength);
```

- Closing the port

```
commPort.Close();
commServer.Close();
```

Content

Symbian OS – Communications overview

Serial communications

→ Socket based communication

Communications Database

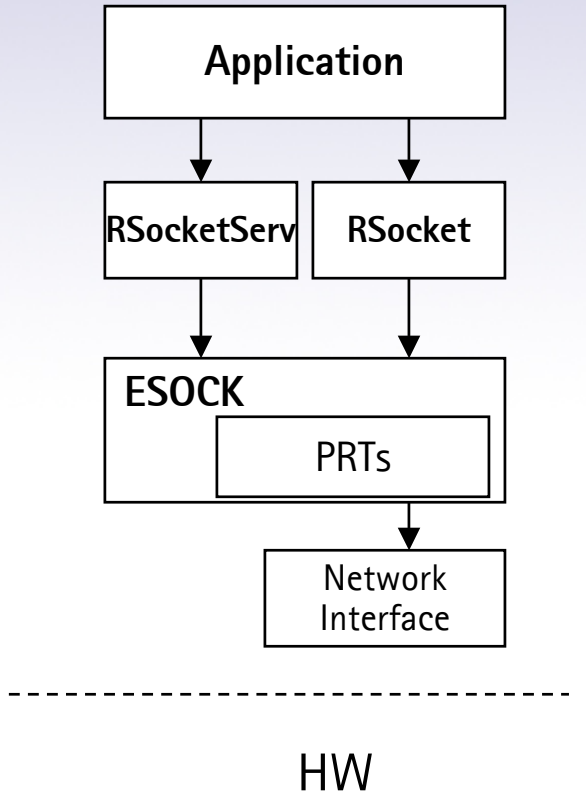
Bluetooth

More information

Get Connected

Socket based Communication

- Symbian OS sockets are similar in concept of BSD UNIX socket
- The socket architecture provides generic client interface and server, where protocol modules can be plugged in.
- Sockets can be used over TCP/IP, IrDa and Bluetooth.
- ESOCK provides RSocketServ and RSocket API.



Socket based Communication

- The RSocketServ class represents an application's session with the socket server.
- RSocketServ lets a client application to query the socket server to determine the number of protocols it knows about and return information about each of these protocols.

```
TInt NumProtocols(TUint& aCount);
TInt GetProtocolInfo(TUint anIndex,TProtocolDesc& aProtocol);
```

- A client application wishing to use sockets will need to have one instance of an RSocketServ object, which will be used to establish a session with the socket server.

```
TInt Connect();
```

- A client application may wish to pre-load protocol modules rather than allowing the socket server to load the modules on demand.

```
void StartProtocol(TUint aFamily,TUint aSockType,TUint aProtocol,
TRequestStatus& aStatus);
```

Socket based Communication

- The RSocket class represents a sub-session of a parent RSocketServ session with the socket server. Before any of the services provided by RSocket can be used, the socket must be open.

```
TInt Open(RSocketServ& aServer,TUint addrFamily,TUint sockType, TUint protocol);
```

  - aServer associates the new socket with the parent session
  - The Internet address family specified by the constant identifier `KAfInet; KIrdaAddrFamily; KBTAddrFamily`
  - Sockets can either operated in a connected or connectionless mode (streams or datagrams). The sockType parameter is used to indicate type of socket to be created: `KSockDatagram or KSockStream.`
  - The protocol module TCPIP.PRT implements a number of protocols, such as UDP and TCP for example. `KProtocolInetUdp` for UDP and `KProtocolInetTcp` for TCP. One can leave the selection of the protocol to the socket server by specifying the `protocol` as `KUndefinedProtocol`. The socket server will use the natural choice of protocol for the selected socket type: `KSockDatagram` with `KProtocolInetUdp` and `KSockStream` with `KProtocolInetTcp`.

- RSocket provides many services including:
  - Connection services, as a client or a server,
  - Setting or querying own address, or querying remote address,
  - Reading data from the socket,
  - Writing data to the socket.

Connection Management

- Differences between Symbian OS 6.1 and 7.0s
  - V6.1
    - Connection management is implemented by `RNif, RGenericAgent`
  - V7.0s
    - Connection Management API is implemented by `RConnection`
    - Supports multi-homing, which allows multiple network interfaces to be active simultaneously
- The API provides interfaces for creating and managing connections
- Using RConnection for
  - Opening and closing a connection
  - Starting a connection, which means associating it with a new underlying interface
  - Attaching the **`RConnection`** instance to an existing interface
  - Obtaining progress information and notification during connection start-up
  - Notifying when sub-connections come up and go down
  - ...

Connection setup

- Implicit connection start-up

```
RSocketServ ss;
RSocket socket;
TRequestStatus status;

// Connect to the socket server
ss.Connect();

// Open a TCP socket
socket.Open(ss, KAfInet, KSockStream, KProtocolInetTcp);

_LIT(KRasAddr,"10.159.24.13");
const TInt KEchoPort = 7;

TInetAddr destAddr;
destAddr.Input(KRasAddr);
destAddr.SetPort(KEchoPort);

// Request the Socket to connect to the destination (implicit connection)
socket.Connect(destAddr, status);
```

Connection setup

- Explicit connection start-up

```
RSocketServ ss;
RConnection exConn;
TRequestStatus status;
_LIT(KRasAddr,"10.159.24.13");
const TInt KEchoPort = 7;

TInetAddr destAddr;
destAddr.Input(KRasAddr);
destAddr.SetPort(KEchoPort);

// Connect to the socket server
ss.Connect();

// Open an RConnection object.
exConn.Open(ss);

// Create overrides
TCommDbConnPref prefs
prefs.SetDialogPreference(ECommDbDialogPrefDoNotPrompt);
prefs.SetDirection(ECommDbConnectionDirectionOutgoing);
prefs.SetIapId(4);

// Start an Outgoing Connection with overrides
exConn.Start(prefs);
```

```
// Open a Socket associated with the connection
RSocket socket;
socket.Open(ss, KAfInet, KSockStream, KProtocolInetTcp, exConn);

// Request the Socket to connect to the destination
socket.Connect(destAddr, status);
```

Secure Socket Communication

- Currently supports two secure sockets standards
  - SSL (Secure Socket Layer) v3.0
  - TLS (Transport Layer Security) v1.0
- Differences between Symbian OS 6.1 and 7.0s
  - V6.1
    - Lack of notification when the secure handshake completes
    - Secure sockets can be set even before the socket is connected
  - V7.0s
    - New classes to handle secure sockets
    - A socket handle must be connected before it is secured

Secure Socket Communication

- Series 60 1.x (Symbian OS 6.1)
  - Set to use SSL secure socket via Rsocket::SetOpt()

```
RSocketServ ss;
RSocket socket;
ss.Connect();
socket.Open(ss, KAfInet, KSockStream, KProtocolInetTcp);
socket.SetOpt(KSoSecureSocket, KSolInetSSL);
socket.Connect(...);
```

  - Set to use TLS secure socket via Rsocket::SetOpt()

```
...  // socket.Connect();
const TInt KNoOfCipher = 4;
const TInt KArraySize = KNoOfCipher * 2;
const TUint8 array[KArraySize] =
{    0x00, 0x0a,
     0x00, 0x04,
     0x00, 0x16,
     0xff, 0xff    };
TPtrC8 ptr(array, KArraySize);
socket.SetOpt(KSoCurrentCipherSuite, KSolInetSSL, ptr);
socket.Connect(...);
```

Secure Socket Communication

- Series 60 2.x (Symbian OS 7.0s/8.0a)
  - Set to use secure sockets via RSocket::SetOpt()

```
RSocketServ ss;
RSocket socket;
ss.Connect();
socket.Open(ss, KAfInet, KSockStream, KProtocolInetTcp);
socket.Connect(...);
socket.SetOpt(KSoSecureSocket, KSolInetSSL);
```

  - Set and use secure sockets with CSecureSocket class

```
...  // socket.SetOpt(KSoSecureSocket, KSolInetSSL);
CSecureSocket tlsSocket;
_LIT(KTLS1,"TLS1.0");
tlsSocket = CSecureSocket::NewL(socket, KTLS1);
// start the handshake
TRequestStatus status;
tlsSocket->StartClientHandshake(status);
User::WaitForRequest(status);
```

Content

Symbian OS – Communications overview

Serial communications

Socket based communication

Communications Database

Bluetooth

More information

Communication database

- Comms Db stores information about ISPs, modem and GPRS settings, WAP, proxies, Bluetooth, connection preferences etc.
- Comms Db is based on Symbian DBMS architecture, hence it is a client/server architecture and provides safe access for multiple clients
- Client access to the database is provided by
  - CCommsDatabase
  - CCommsDbTableView
  - CCommsDbConnectionPrefTableView

Communication database

- To access the comms database for reading values

```
CCommsDatabase* commsDb;
commsDb = CCommsDatabase::NewL();
CleanupStack::PushL(commsDb);

CCommsDbTableView* iapTable = commsDb->OpenTableLC(TPtrC(IAP));
TInt result;
TBuf<40> iapName;
TUint32 iapId;
// Walk through records
result = iapTable->GotoFirstRecord();
while (result == KErrNone)
    {
    iapTable->ReadTextL(TPtrC(COMMDB_NAME), iapName));
    iapTable->ReadUintL(TPtrC(COMMDB_ID), iapId));
    result = iapTable->GotoNextRecord();
    }
CleanupStack::PopAndDestroy(2); //iapTable, commsDb
```

Communication database

- To access the comms database for writing or modifying values

```
CCommsDatabase* commsDb;
commsDb = CCommsDatabase::NewL();
CleanupStack::PushL(commsDb);

TBufC<32> commsdb_name_val(_L("my company"));
TUint32 network_id;
commsDb->BeginTransaction();

// creating a new network record
CCommsDbTableView* network = commsDb->OpenTableLC(TPtrC(NETWORK));
network->InsertRecord(network_id);
network->WriteTextL(TPtrC(COMMDB_NAME), commsdb_name_val);
network->PutRecordChanges();
CleanupStack::PopAndDestroy(network);

err= commsDb->CommitTransaction();
CleanupStack::PopAndDestroy(); // commsDb
```

Communication database

Step to step creating a new IAP record

1. Create a Network record with the name is the name of the new IAP
2. Create an OUTGOING_GPRS record with the name is the name of the new IAP
3. Create a WAPAccessPoint with the name is the name of the new IAP and with at least the current bearer column
4. Create a new IAP record and refer the outgoing GPRS and the network records
5. Create a WAPIPBearer with the access point Id to point to the WAPAccessPoint record above

Communication database

Deprecated functions Symbian OS 7.0s

- All agent-specific functions
- `GetCurrentDialOutSettingL(const TDesC& aSetting, TUint32& aValue)`
- `GetCurrentDialInSettingL(const TDesC& aSetting, TUint32& aValue)`

Incorrect reference

- IAP_BEARER_TYPE

```
S60 SDK v2.0/2.1: Type: TUint32. Null yes
S60 SDK v2.2: Type: Text. Null: yes
Correct type: Type: Text. Null: No
CCommsDbTableView::WriteTextL(TPrtC(IAP_BEARER_TYPE), TPrtC(MODEM_BEARER))
```

Missing reference

- Timeout values for modem bearers

```
#define LAST_SOCKET_ACTIVITY_TIMEOUT        _S("LastSocketActivityTimeout")
#define LAST_SESSION_CLOSED_TIMEOUT         _S("LastSessionClosedTimeout")
#define LAST_SOCKET_CLOSED_TIMEOUT          _S("LastSocketClosedTimeout")
CCommsDbTableView::WriteUintL(TPrtC(LAST_SOCKET_ACTIVITY_TIMEOUT), 180)
```

Content

Symbian OS – Communications overview

Serial communications

Socket based communication

Communications Database

Bluetooth

More information

Bluetooth Basis

- Wireless technology for short-range communications; most typically Class 2 devices (10 meters range)
- Uses 2.4GHz license-free ISM (Industrial, Scientific, Medical) band
- Primary goals of the technology
  - cheap
  - small
  - low power consumption

  -> ideal for mobile phones

Bluetooth Profiles

- Guarantee interoperability:
  - A Bluetooth profile implements a certain use case in an interoperable way using certain Bluetooth protocols in a determined way
- Some of current Bluetooth profiles:
  - Generic Access Profile: the building block for other profiles. Dealing with Link Manager Protocol and Baseband Link Controller layers of the BT stack and focus on discovery
  - Serial Port Profile: defines the protocols and procedures to be used by devices using BT for serial cable emulation
  - Dial-Up Networking Profile
  - Fax Profile
  - Generic Object Exchange Profile: defines to run OBEX over RFCOMM. Dealing with the transfer of objects over BT
  - Object Push Profile
  - File Transfer Profile

Bluetooth Protocols

- OBEX (Object Exchange Protocol):
  - sending and receiving files (Object Push)
  - remotely browsing and accessing files (FTP)
  - Session-based protocol
  - Adopted from IrDA specification
- RFCOMM
  - Serial port (RS-232) emulation
  - Data stream
- L2CAP (Logical Link Control and Adaptation Protocol)
  - Segments and reassembles data packets between higher-level protocols and HCI
  - Multiplexes data from higher-level protocols
  - A fundamental protocol for accessing most Bluetooth services

Bluetooth Protocols

- SDP (Service Discovery Protocol)
  - Provides methods for entering Bluetooth service information into service DB and for accessing a remote device's service DB
- HCI (Host Controller Interface)
  - Access to link manager and hardware for low-level configurations
  - Get information on the local Bluetooth device using HCI commands (specified in the Bluetooth v1.1 Core)
- LMP (Link Manager Protocol)
  - Link setup (authentication and encryption, control and negotiation of baseband packets) between devices
  - Power modes and connection states of a Bluetooth unit

Bluetooth Glossary

- Roles:
  - Master : device typically searching for devices and establishing the connection
  - Slave : device typically waiting for connections
- Links:
  - ACL Asynchronous Connectionless links for data
    - Point-to-point (1 master and 1 slave)
    - Point-to-multipoint (1 master and up to 7 slaves); no direct connections between slaves (only via the master)
  - SCO Synchronous Connection-oriented links for audio, only point-to-point
- Packets:
  - ACL packets can be 1-3-5 slots long (bigger payload in 3/5-slot-packets, as the header and guard time only once, but bigger risk for interference)
  - SCO packets are only one-slot long (synchronous)

Bluetooth API Modules in Symbian OS

- Bluetooth Sockets
  - Encapsulate access to **RFCOMM** and **L2CAP** through a TCP/IP-like sockets interface
- Service Discovery Database
  - Encapsulates one side of **SDP**: a local service uses it to record its attributes to be discovered by remote devices
- Service Discovery Agent
  - Encapsulates the other side of **SDP**: it allows you to discover the services and attributes of remote devices
- Bluetooth Security Manager
  - Enables services to set appropriate security requirements that incoming connections must meet
- Bluetooth UI
  - Used for calling a dialog that asks users for remote device selection

Bluetooth Connection and Communication

- Start service advertisement or device and service discovery
  - Server to start service advertisement
    - Service advertisement is done via the Bluetooth service discovery “server”, which provides a session to the service discovery database
  - Client to discover a server device then the services advertised by the server
    - Device discovery is done via either the extended notifier server or the generic host name resolution services
    - Service discovery is done via the Bluetooth service discovery agent
- Set connection security
  - Server to stipulate the security settings
  - Client to implement the server’s security requirements
    - Both client and server can configure security settings via the Bluetooth security server
- Setup connection
  - Server to start listening for incoming connections
  - Client to initiate a connection
    - Both client and server can establish the connection using the generic Symbian OS Socket interfaces
- Communication
  - Client and server can exchange data over an open session via a connected socket

Bluetooth API

- Service advertisement

```
RSdp sDs;
RSdpDatabase sDdb;
TSdpServRecordHandle serviceRec;
sDs.Connected();
sDdb.Open(sDs);
// start advertizing by creating a new service record
const TUint KServiceUID = KMyAppUid; // e.g. 0x101ABABA
sDdb.CreateServiceRecordL(KServiceUID, serviceRec);
// create service record attributes

// update record attributes

// delete the record when no longer needed
// close the session when no longer needed
```

Bluetooth API

- Device Discovery:

```
RNotifier, RHostResolver
KDeviceSelectionNotifierUid
```

  - The RNotifier class presents the user with a UI dialog listing all Bluetooth devices currently in range and advertising appropriate services.
  - The RHostResolver API provides a way for the client application to programmatically search for local devices

```
TBTDeviceSelectionParams, TBTDeviceSelectionParamsPckg
TBTDeviceResponseParams, TBTDeviceResponseParamsPckg
```

  - Useful functions

```
TBTDeviceSelectionParams::SetUUID(const TUUID& aUUID);

TBool TBTDeviceResponseParams::IsValidDeviceName())
TDesC TBTDeviceResponseParams::DeviceName();
TBool TBTDeviceResponseParams::IsValidBDAddr();
TBTDevAddr& TBTDeviceResponseParams::BDAddr();
```

Bluetooth API

- Service discovery

  CSdpAgent, MSdpAgentNotifier

  TBTDevAddr

  - Pure virtual functions from MSdpAgentNotifier

```
AttributeRequestComplete(TSdpServRecordHandle aHandle,
                         TInt aError) = 0;
AttributeRequestResult(TSdpServRecordHandle aHandle,
                       TSdpAttributeID aAttrID,
                       CSdpAttrValue* aAttrValue) = 0;
NextRecordRequestComplete(TInt aError,
                       TSdpServRecordHandle aHandle,
                       TInt aTotalRecordsCount) = 0;

CSdpAgent* NewL(MSdpAgentNotifier& aNotifier, const TBTDevAddr& aDevAddr);
```

Bluetooth API

- Security settings

```
RBtMan, RBTSecuritySettings, TBTServiceSecurity
// 1. Create and open session to the security manager
RBTMan secMan;
secMan.Connect();
// 2. Create and open a subsession
RBTSecuritySettings secmanSubSession;
secmanSubSession.Open(secMan);
// 3. Security settings
TBTServiceSecurity serviceSec(KMyAppUid, KSolBtRFCOMM, 0);
serviceSec.SetAuthentication(ETrue);
serviceSec.SetEncryption(ETrue);
serviceSec.SetAuthorisation(ETrue);
serviceSec.SetChannelID(iChannel);
secmanSubsession.RegisterService(serviceSec, iStatus);
// 3. Cleanup
secmanSubSession.Close();
secMan.Close();
```

Bluetooth API

- Connection establishment
  - Server (slave)

```
RSocketServ, RSocket
_LIT(KL2Cap, "L2CAP"), _LIT(KRfComm, "RFCOMM");
RSocketServ socketServ;
RSocket btSocket;
socketServ.Connect()
btSocket.Open(socketServ, KRfComm);
TInt channel;
btSocket.GetOpt(KRFCOMMGetAvailableServerChannel, KSolBtRFCOMM, channel);
TBTSockAddr addr;
addr.SetPort(channel);
btSocket.Bind(addr));
btSocket.Listen(2);
RSocket accept;
accept.Open(socketServ);
btSocket.Accept(accept,iStatus);
SetActive();
```

Bluetooth API

- Client (master)

```
RSocketServ, RSocket
_LIT(KL2Cap, "L2CAP"), _LIT(KRfComm, "RFCOMM");
RSocketServ socketServ;
RSocket btSocket;
socketServ.Connect()
btSocket.Open(socketServ, KRfComm);
btSocket.Connect(btAddr, iStatus);
SetActive();
```

- Data exchange

```
btSocket.Read();
btSocket.Write();
```

Content

Symbian OS – Communications overview

Serial communications

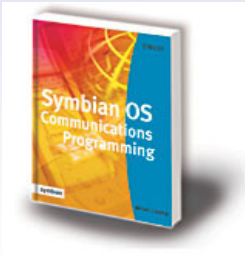
Socket based communication

Communications Database

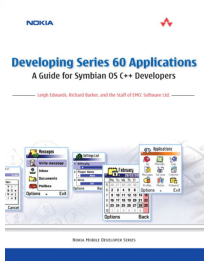
Bluetooth

More information

More Information



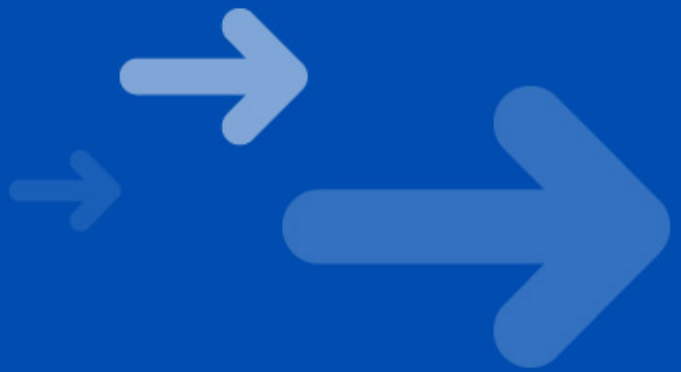
- Symbian OS Communications Programming
  - Specifically about communications in Symbian OS



- Developing Series 60 Applications
  - The Official Nokia Guide to mobile application development on the Series 60 Platform.

Get Connected

Thank You



Get Connected